Auditing Verifiable Instruction-Following Benchmarks: Specification Gaps, False Verdicts, and Cross-Benchmark Reliability

Anonymous authors (draft for internal review; not submitted)

Draft of 14 October 2026

Abstract

Verifiable instruction-following benchmarks (IFEval, IFBench) and the reward functions built from them (IF-RLVR) score a response with a short program per constraint. We audit three verifier collections with one protocol: the 25 IFEval templates, the 58 IFBench test templates, and 29 open-instruct IF-RLVR training templates, all at pinned upstream commits and without modifying official code. We measure how often responses that contain no task content satisfy the checks, build independent literal-definition oracles for 56 of the 112 templates, and test metamorphic relations. A fixed response (`Yes.`) satisfies every constraint of 11.3% of IFEval prompts, 34.7% of IFBench test prompts and 7.4% of sampled IF-RLVR training prompts; a response written for a different prompt is accepted by all 62 evaluable templates; an empty response is never accepted. We separate this by-design content blindness from implementation defects, and report five reproducible defect candidates (one already documented for IFEval) plus several specification-dependent ambiguities. Our contributions are measurements, a taxonomy, an oracle and regression-test suite, and an open toolkit; we do not claim that auditing verifiers is new, nor that content blindness is a bug. All adequacy-related context uses AI raters from one model family and is not independently validated.

1 Introduction

Rule-based verifiers have become the default way to evaluate and to train instruction following: IFEval [18] introduced 25 verifiable instruction types, IFBench [10] added 58 out-of-domain constraints and the IF-RLVR training recipe, and several post-training pipelines use IFEval/IFBench-style verifiers as reward or as data filters [8, 9]. A verifier is software, and its verdict is only as good as its specification and implementation. Three kinds of question follow. (i) *Is the verdict correct for the stated constraint?* (ii) *What does a passing verdict guarantee about the response?* (iii) *Do the answers hold across the benchmark and the training-time implementation?*

Prior work already shows that these verifiers are imperfect and that constraint satisfaction can come apart from task fulfilment: IFEval's authors note that loose scoring is likely to introduce false positives [18]; IFBench reports constraint over-optimisation and proposes a reward-model gate [10]; IFDecorator catalogues hacking patterns and adds an LLM intent check [5]; IFEval++ revises flawed IFEval test cases and checker/argument mismatches [4]; and the IFBench maintainers and community fix verifier bugs through issues and pull requests, most recently PR #34, which corrected six IFBench templates [1]. We therefore do *not* claim novelty for auditing verifiers or for the existence of constraint-over-task behaviour.

This paper contributes a different, narrower thing: a reproducible comparison of three verifier collections under one protocol, with deterministic measurements that do not depend on any LLM judge. Concretely:

1. a pinned, scripted comparison of the IFEval, IFBench and open-instruct IF-RLVR verifier collections, including their provenance differences (Sec. 3);
2. an evaluation of *prompt-independent constraint acceptance*: content-free probes, wrong-prompt responses and structure-preserving transformations at template, instance and prompt level (Sec. 5);
3. independent-oracle coverage (56 of 112 templates), metamorphic and property-based tests, with a taxonomy that separates implementation defects from intended invariance, content blindness and ambiguity (Secs. 4 and 5);
4. a list of implementation defects with independently validated minimal examples, and of specification ambiguities (Sec. 6);
5. an open verifier-audit toolkit and regression-test suite (Research-artifact availability statement and Appendix C).

Section 7 gives secondary context on what the measurements imply for RL-trained small models; it supports no optimizer claim.

2 Related Work

Verifiable instruction-following benchmarks. IFEval [18] and IFBench [10] are the two collections we study; open-instruct's IFEvalG module [2] holds the training-time constraint implementations (the IF-RLVR data). Wording robustness of verifiable counts is studied by WISE [17].

Audits of instruction-following verifiers and test cases. IFEval++ [4] identifies four flaw categories in IFEval (checker implementation, argument implementation, prompt typos, unachievable multi-constraint cases) and measures reliability on revised cousin prompts; it covers IFEval only. The IFBench tracker and PR #34 [1] fix verifier bugs in six IFBench templates (word versus character indices, punctuation tokens, substring counts, a missing check); the open-instruct tracker shows an active stream of IFEvalG fixes and a concurrent report on 14 October 2026 [2]. Dynamic Boundary Evaluation bench-tests its own re-implementation of the 25 IFEval types against upstream and then relies on upstream checkers [13]. Instruction Stacking Collapse validates its own 19 verifiers against hand labels formed blind to the verdict [3].

Verifier testing and gaming in RLVR. Fuzzing and metamorphic testing of reward verifiers have been developed for math, JSON and code verifiers [11, 16]; isomorphic perturbation testing exposes verifier gaming in inductive reasoning [7]. For instruction following, UNSPECIFIC shows that synthesised constraints can be trivially satisfied by copy-and-paste [12]; IFDecorator [5], AdvancedIF [6] and IF-RewardBench [15] address judge- or rubric-side mitigation, and Wei et al. audit verifier-passed IF-RLVR rollouts with a different-family judge and report that most are superficial shortcuts at the final checkpoint [14]. We apply the testing techniques (probes, metamorphic relations, oracles) to the three IF collections; the techniques themselves are not new.

Positioning. Relative to this work we provide (a) one protocol over three collections including the training-time implementation, (b) deterministic prompt-independent acceptance numbers, and (c) oracles and regression tests released as a toolkit. A documented, single-reviewer search (726 database records, 35 included studies, 14 October 2026) found no study doing (a)–(c) together; this is not a claim of priority.

3 Verifier Collections

IFEval-25: 541 prompts, 834 constraint instances, 25 templates, scored with the classic classes vendored in IFBench (AST-identical to the upstream Google implementation for all 29 common classes). **IFBench-58**: the 300 test prompts, 344 instances, 58 templates. **IF-RLVR-29**: the 29 open-instruct IFEvalG templates not in IFEval; the public IF_multi_constraints_upto5 dataset has 95,373 rows and 248,446 constraint instances; we use up to 120 sampled instances per template (3,480) and, for prompt-level rates, 3,000 sampled training prompts scored with all their constraints by the training verifier path (strict only). Pins: IFBench `1c40f0c1` (contains PR #34), open-instruct `11826255` (the upstream heads on 14 October 2026), google-research/instruction_following_eval `e6890f85`. Provenance: open-instruct's IFEvalG differs at AST level from the evaluation-time classes in 8 IFEval classes (mostly formatting-level differences; behavioural equivalence was not tested).

4 Verifier Specification and Audit Methodology

Specification. For each template the *specification* is the instruction text plus the stored arguments. A verdict is *correct* if it agrees with a literal reading of that specification; an *ambiguity* arises when two reasonable readings disagree. Narrow checks deliberately ignore most of the response, so acceptance of a response that does not address the task is not by itself a defect.

Probes and transformations. Content-free probes: `Yes.`, `No.`, `.`, one word, an emoji flood, a word flood, numbers, digits, a generic sentence; the empty response; an echo of the prompt. Transformations of natural passing responses: whitespace (F_ws), lower-casing (F_case), a polite prefix (F_prefix), word shuffle within lines (D_shuffle), same-length lexical replacement (D_lex), and a response written for a different prompt with identical arguments (X_swap). F transformations preserve structure and need not preserve the verdict; D transformations destroy content while keeping surface structure.

Round-trip. Every instance is rebuilt from its stored arguments; a changed argument means the verifier checks something other than the prompt states.

	IFEval-25	IFBench-58	IF-RLVR-29
Templates accepting a probe on ≥90% of instances	3/25 (12%)	22/58 (38%)	10/29 (34%)
Instances passed by `Yes.`	189/834 (22.7%)	126/344 (36.6%)	1029/3480 (29.6%)
Instances passed by the empty response	0/834	0/344	0/3480
Prompts passed by `Yes.` (all constraints)	61/541 (11.3%)	104/300 (34.7%)	221/3000 (7.4%)
Prompts passed by any content-free probe	99/541 (18.3%)	137/300 (45.7%)	368/3000 (12.3%)
Templates accepting another prompt's response	19/19	26/26	17/17

Table 1: Prompt-independent acceptance (strict). Denominators in the cells; IF-RLVR prompt rates use 3,000 sampled training prompts. Wilson 95% intervals are in `docs/reports/m5c1_cross_benchmark_tables.md`, e.g. `Yes.`: [8.9, 14.2], [29.5, 40.2], [6.5, 8.4]. "Another prompt's response": templates with ≥10 natural passing instances whose verdict is kept in ≥95% of cases.

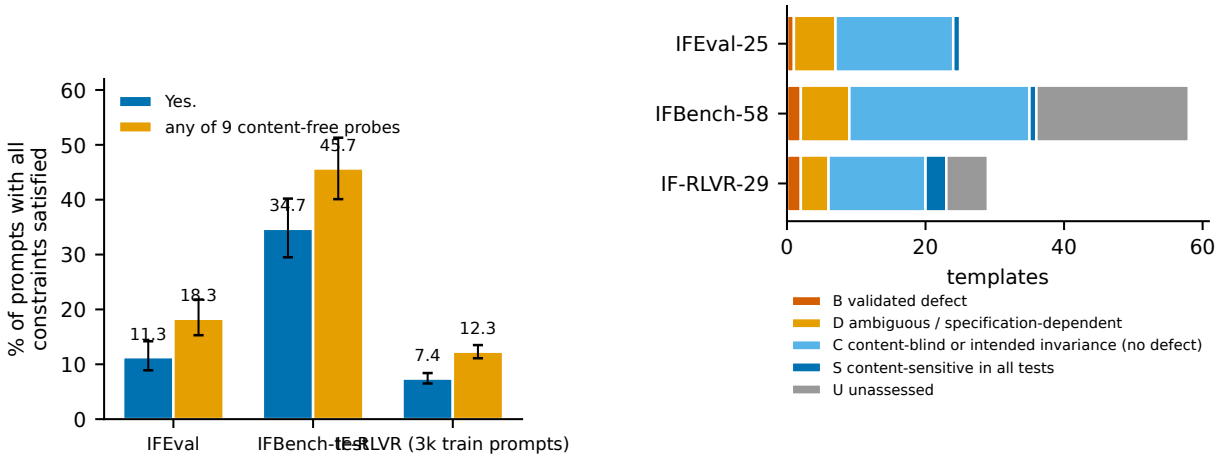


Figure 1: Left: share of prompts whose constraints are all satisfied by `Yes.` or by any of nine content-free probes (Wilson 95% intervals). Right: template classes per collection (B validated defect; D ambiguous; C content-blind or intended invariance; S; U).

Oracles, metamorphic relations, property tests. For 56 templates we wrote independent oracles: short re-expressions of the instruction text using different primitives than the checker (no shared tokeniser or library call). Independence is limited: for 17 of them the author read the official code first, and wording choices are the author's. Each oracle is compared with the verifier on probes, constructed positives, natural responses and 400 random property-based strings. *Must-preserve* relations (appending whitespace; lower-casing for case-insensitive templates) and *must-change* relations (a minimal edit that makes an oracle-true response violate the instruction) are scored separately.

Classes and denominators. Templates are labelled A (intended invariance), B (validated implementation defect), C (task-content blindness), D (ambiguous or specification-dependent), S (content-sensitive in every test) or U (not assessed beyond the probes). Three denominators are never mixed: *templates*, *constraint instances* and *prompts* (all constraints of a prompt must pass). Proportions use Wilson 95% intervals.

5 Cross-Benchmark Results

Prompt-independent acceptance. Table 1 and Fig. 1: `Yes.` alone satisfies all constraints of 61/541 (11.3%) IFEval prompts, 104/300 (34.7%) IFBench test prompts and 221/3,000 (7.4%) sampled IF-RLVR training prompts; the union of the nine content-free probes reaches 18.3%, 45.7% and 12.3%. At instance level `Yes.` passes 22.7%, 36.6% and 29.6%. At template level 3/25, 22/58 and 10/29 templates accept at least one content-free probe on ≥90% of their instances. The empty response is accepted in 0% of instances in all three collections. Loose and strict scoring coincide for the probes. The collections are not comparable by these rates alone because their constraint mixes differ.

Transformations. Fig. 2: whitespace changes almost never alter a passing verdict (99.7–100%), a response from another prompt keeps its verdict in 99.9–100% of cases (all verifiers are prompt-independent given their

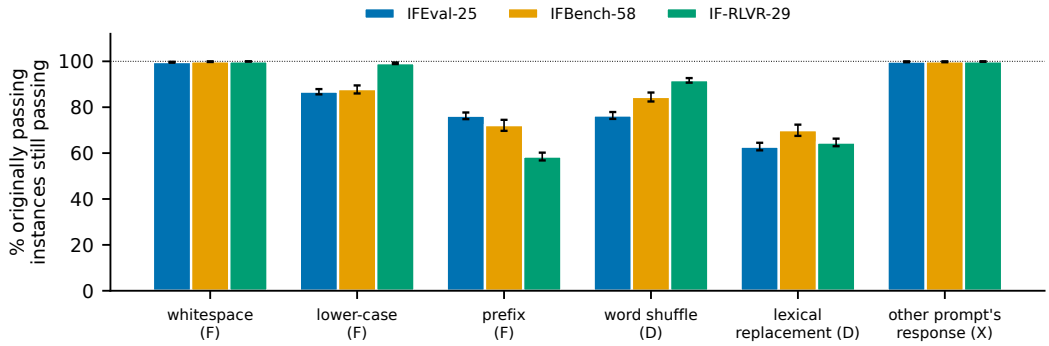


Figure 2: Share of originally passing natural instances that still pass after each transformation (pooled over templates, strict; Wilson 95% intervals). F: structure-preserving; D: content-destroying; X: response written for another prompt with identical arguments.

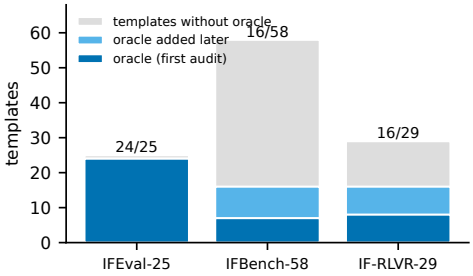


Figure 3: Templates with an independent oracle.

arguments), and word shuffling keeps 76.4%, 84.5% and 91.8% of passing instances (14/25, 22/34 and 14/22 evaluable templates retain ≥95%), versus 62.8%, 70.0% and 64.6% for lexical replacement. Content-sensitive templates (e.g. repeat/copy constraints) are the exceptions.

Oracle audit. With the 39 oracles of the first audit (49,538 comparisons, 954 disagreements, all within the adjudicated defect/ambiguity groups) and 17 new ones (16,372 comparisons, 1,264 disagreements), coverage is IFEval 24/25, IFBench 16/58, IF-RLVR 16/29 templates (Fig. 3). Table 2 lists the 17 new oracles; 4 templates agree on all checks, 11 differ for interpretation reasons (tokenisation of punctuation-glued tokens, sentence segmentation, one-character palindromes), and two reveal defects (Sec. 6). Among 3,490 must-preserve checks one template violates the relation (`format:line_indent`, 99/99); among 2,144 must-change checks one template fails to react (`detectable_format:bigram_wrapping`, 86/86; classified as ambiguity). 56 templates remain without oracle (library-dependent segmenters and lists, prompt-dependent extraction, single-instance custom tasks); our estimate of a realistic ceiling is about 70/112.

6 Confirmed Implementation Defects and Specification Ambiguities

Table 3 lists what survived reproduction on the latest upstream revisions (14 October 2026), a duplicate search of all issues, pull requests and commits of both repositories, and an independent oracle. None of the four newer items is touched by IFBench PR #34. Reports to the maintainers are prepared but not yet submitted; we therefore present them as reproducible behaviours, not as new bugs, until maintainers respond. Further ambiguities (class D) include: `english_capital/lowercase` and `response_language` depend on an unseeded language detector; `postscript` accepts the marker anywhere in a line; sentence-level templates are vacuous on punctuation-free lines; `keywords:palindrome` counts one-character tokens; `first_word_answer` rejects a first token with attached punctuation or markdown; `bigram_wrapping` ignores a trailing odd word. Loose scoring is intentionally lenient. Appendix A gives minimal examples.

Collection / template	Oracle conf.	Checks	Off. pass/Orc. fail	Off. fail/Orc. pass	Must-change misses
IFBench-58 / words:no_consecutive	high	780	12	3	0/136
IFBench-58 / words:alphabet	high	700	1	0	0/122
IFBench-58 / words:vowel	medium	860	0	0	0/77
IFBench-58 / format:title_case	low	620	0	0	–/–
IFBench-58 / format:line_indent	high	820	0	11	0/99
IFBench-58 / words:palindrome	medium	740	0	0	0/60
IFBench-58 / words:last_first	medium	660	0	23	0/162
IFBench-58 / words:paragraph_last_first	high	700	3	1	0/108
IFBench-58 / format:sub-bullets	high	892	345	0	0/73
IF-RLVR-29 / detectable_format:bigram_wrapping	medium	1200	254	0	86/86
IF-RLVR-29 / keywords:no_adjacent_consecutive	medium	1200	0	41	0/262
IF-RLVR-29 / keywords:palindrome	low	1200	482	4	0/88
IF-RLVR-29 / keywords:start_end	medium	1200	48	5	0/114
IF-RLVR-29 / first_word:first_word_answer	medium	1200	0	25	0/284
IF-RLVR-29 / last_word:last_word_answer	high	1200	0	1	0/211
IF-RLVR-29 / paragraphs:paragraphs	high	1200	0	5	0/184
IF-RLVR-29 / count:count_increment_word	high	1200	0	0	0/78

Table 2: The 17 oracles added in the second audit (official strict verifier vs. oracle). "Checks" = probes, constructed positives, natural responses and 400 random strings; counts are not natural rates. "Must-change misses" = oracle-true responses edited to violate the instruction that the verifier still passes.

Template	Behaviour (observed vs. expected)	Footprint	Type / prior report
IF-RLVR `copy_span_idx`, n_{start}=0	`if not n_start`: a random start replaces 0; the correct span is accepted 4.8% of the time across 25 fresh processes (expected 100%), wrong spans by chance	21 of 1,815 instances (1.2%); 21 of 95,373 rows	defect (nondeterministic); no prior report found
same, end index	prompt says "between (and including) index n_{start} and n_{end}"; code slices $[n_{start} : n_{end}]$; IFBench's sibling class is inclusive	readings differ for 1,523/1,815 instances	*ambiguity* (clarification request)
IF-RLVR `exclude_word_harder`	keyword detected only between two literal spaces: missed at response start/end and next to newline or tab; expected: any whole-word occurrence is a violation	5,182 instances; 9/445 (2.0%) passes in our sampled rollouts contain the keyword token (28 = 6.3% case-insensitively)	defect for boundary positions; case/punctuation adjacency is *interpretation-dependent*; none found
IFBench `line_indent`	blank lines removed while iterating: a valid staircase with two consecutive blank lines (e.g. trailing `\n\n`) is rejected	9 test prompts; 0/261 saved responses changed	defect (false negative); none found
IFBench `sub-bullets`	a response with no bullet passes (vacuous truth) although it must include bullets	12 test prompts; 2 of 18 passes in saved responses	defect (false positive); none found
IFEval `letter_frequency`	non-a–z target (e.g. `#`) replaced by a random letter at build time	2/33 instances	defect (nondeterministic); documented for IFEval by Dong et al. [4]

Table 3: Defects and the main ambiguity. "Saved responses" are our own re-scoring of 29 sets of outputs of small models on IFBench test (aggregate counts only). Severity is low in every case; none can explain an aggregate benchmark trend.

7 Secondary Context: Does Content Blindness Matter for RL?

We re-used earlier runs of a small policy (Qwen3.5-0.8B) trained with IF-RLVR rewards on the public IF-RLVR training prompts and evaluated on IFBench test, and rated a stratified sample of 230 natural responses with four AI raters from one model family (Fleiss κ 0.86 for binary adequacy; no human and no different-family rater, so agreement with humans is unknown). Official loose accuracy rose from 20.3% at baseline to 25.6–29.9% after 64 updates while the AI-rated share of responses that both pass the verifier and engage with the task did not exceed the baseline's 16.3% (1.7% and 8.5% for the two optimizers at step 64; 10–18 passes per cell, wide intervals). This is exploratory, from one model and one benchmark; the direction agrees with published reports [5, 10, 14]; the optimizer difference is not interpreted (three seeds). The deterministic findings above do not depend on these ratings.

8 Threats to Validity

Single adjudicator and author-written oracles: classification of ambiguities and oracle choices are the author's; oracles for 17 templates were written after reading the official code. **Coverage:** 56 of 112 templates lack an oracle, so "no defect found" means not found by probes, round-trip, natural-response transformations and metamorphic tests. **Sampling:** IF-RLVR rates use sampled instances and prompts; natural passing pools for many IFBench templates are small (34 of 58 templates had $\geq$10). **Versions:** results hold for the pinned commits; IFBench scores computed before PR #34 are not comparable. **Environment:** fresh-interpreter, not container, reproduction; an unseeded language detector affects class-D templates. **Prior-art search:** documented but single-reviewer and API-based; no claim of priority. **Judges:** adequacy labels come from

one model family and are not human ground truth; an independent-family judge was prepared but not run. **Test-set reuse:** IFBench test responses were used only to validate measurements.

9 Practical Recommendations

(1) Report, next to every verifier-based score, the pass rate of content-free probes for the same prompts (here 11–46% of prompts) as a floor. (2) Pin verifier revisions and hashes; re-score before comparing results across upstream fixes. (3) Treat training-time and evaluation-time implementations as different artifacts until equivalence is tested. (4) Add vacuity guards ("response must contain the structure") and boundary tests to new verifiers; run the released regression suite and oracles in CI. (5) When claiming task quality, add a judge or rubric component as prior work proposes [5, 6, 10]; narrow verifiers measure the constraint, not the task. (6) Prefer clarification requests to bug reports where the specification is ambiguous.

10 Conclusion

Under one protocol, verifier collections of IFEval, IFBench and open-instruct IF-RLVR show large, deterministic, prompt-independent acceptance that is mostly by design, together with a small number of reproducible implementation defects and several specification gaps. The contribution is an auditable, open measurement and test suite, not a claim of first discovery.

Limitations

This is an audit of three verifier collections at pinned commits. Defects have low severity and small footprints; reports upstream are pending. Oracle coverage is 50% of templates and partly author-dependent; adequacy context is AI-rated within one model family and covers one small model on one benchmark. No human study was conducted.

Research-artifact availability

Code, regression tests, oracles, candidate fixes, pinned-source manifests, raw and aggregated results and the upstream-report drafts are in the project repository (currently a private Hugging Face repository, `Dhruv1000/qwen35-logra-ifbench`; to be released anonymously for review and publicly after acceptance). Data used: public IFEval, IFBench test and IF_multi_constraints_upto5. Model outputs used only in the secondary context are small-model generations; no credentials or private data are included.

References

[1] Allen Institute for AI and contributors. allenai/IFBench: repository, pull request #34 and issue tracker. `https://github.com/allenai/IFBench`, . Accessed 14 October 2026; pinned commit 1c40f0c10d9b5c5c2f10a175a28007ebb64f7f4d.

[2] Allen Institute for AI and contributors. allenai/open-instruct: repository and issue tracker. `https://github.com/allenai/open-instruct`, . Accessed 14 October 2026; pinned commit 11826255077617a46919ce75cadf1f3d53f30dac.

[3] Atul Anand and Sourav Chattaraj. Instruction stacking collapse: A benchmark and the capability-dependent value of prompt compilation. *arXiv preprint arXiv:2608.02639*, 2026.

[4] Jianshuo Dong, Yutong Zhang, Yan Liu, Zhenyu Zhong, Tao Wei, Chao Zhang, and Han Qiu. Revisiting the reliability of language models in instruction-following. *arXiv preprint arXiv:2512.14754*, 2025.

[5] Xu Guo, Tianyi Liang, Tong Jian, Xiaogui Yang, Ling-I Wu, Chenhui Li, Zhihui Lu, Qipeng Guo, and Kai Chen. IFDecorator: Wrapping instruction following reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2508.04632*, 2025.

[6] Yun He, Wenzhe Li, Hejia Zhang, et al. AdvancedIF: Rubric-based benchmarking and reinforcement learning for advancing LLM instruction following. *arXiv preprint arXiv:2511.10507*, 2025.

[7] Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. LLMs gaming verifiers: RLVR can lead to reward hacking. *arXiv preprint arXiv:2604.15149*, 2026.

[8] NVIDIA. Nemotron 3 nano: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning. *arXiv preprint arXiv:2512.20848*, 2025.

[9] Hao Peng, Yunjia Qi, Xiaozhi Wang, Bin Xu, Lei Hou, and Juanzi Li. VerIF: Verification engineering for reinforcement learning in instruction following. *arXiv preprint arXiv:2506.09942*, 2025.

[10] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. *arXiv preprint arXiv:2507.02833*, 2025.

[11] Jaideep Ray. Before the model learns the bug: Fuzzing RLVR verifiers. *arXiv preprint arXiv:2606.01066*, 2026.

[12] Jeet Sharma, Balpreet Kaur, Jeremiah Hong, Hamed Zamani, and Haw-Shiuan Chang. UNSPECIFIC: General constraint synthesis for breaking copy-and-paste shortcut in LLM instruction following. *arXiv preprint arXiv:2608.09154*, 2026.

[13] Haoxiang Wang, Da Yu, et al. Beyond fixed benchmarks and worst-case attacks: Dynamic boundary evaluation for language models. *arXiv preprint arXiv:2605.06213*, 2026.

[14] Shaohang Wei, Zikun Su, Feifan Song, Wen Luo, Wei Li, Guangyue Peng, and Houfeng Wang. Verifier-induced support reshaping in on-policy optimization. *arXiv preprint arXiv:2608.00220*, 2026.

[15] Bosi Wen, Yilin Niu, Cunxiang Wang, Xiaoying Ling, Ying Zhang, Pei Ke, Hongning Wang, and Minlie Huang. IF-RewardBench: Benchmarking judge models for instruction-following evaluation. *arXiv preprint arXiv:2603.04738*, 2026.

[16] Esther Xin. Where the verifier fails: A category-level audit of reward signals in RLVR. *arXiv preprint arXiv:2609.01354*, 2026.

[17] Qishi Zhan, Seoyeon Jang, Zihan Dong, et al. When verifiable counts depend on wording: Auditing wording robustness in instruction following. *arXiv preprint arXiv:2610.05278*, 2026.

[18] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

A Verifier Examples

Observed official strict verdicts at the pinned commits (reproduce with `docs/upstream/repro_*.py`, `scripts/m5c1_probe_one.py`).

Template	Input	Verdict	Comment
`format:sub-bullets`	`Yes.`	pass	no bullet at all (defect)
`format:line_indent`	valid staircase + `\n\n`	fail	blank-line removal bug (defect)
`exclude_word_harder`, kw `coast`	`the coast is calm` / `calm coast` / `coast`	fail / pass / pass	boundary cases pass (defect)
`copy_span_idx`, n_{start}=0	correct span `prompt[0:9]`	pass in ≈2% of rebuilds	random start (defect)
`keywords:forbidden_words`	any response lacking the words	pass for all 9 probes	absence-type constraint (class C)
`punctuation:no_comma`	`Yes.`	pass	absence-type constraint (class C)
`words:prime_lengths`	`.`	pass	vacuous on no words (class D)
`sentence:increment`	three unpunctuated equal-length lines	pass	one "sentence" (class D)
`english_lowercase`	`ok then` (300 repeats)	varies	unseeded language detection (class D)
`detectable_content:postscript`	`I cried. P.S. sections overrated`	pass	marker anywhere in a line (class D)

B Secondary Context Table

Official loose accuracy and AI-rated joint success (task engagement and constraint satisfaction) as % of all 300 responses; joint success post-stratified from 230 rated natural items, item-bootstrap 95% intervals; baseline 1 run, other cells 3 seeds. Descriptive only; no optimizer comparison is made.

	official loose	task engagement (T)	joint (C and T)
baseline	20.3	64.1 [44.1, 84.0]	16.3 [12.2, 20.3]
RL, step 64, optimizer 1	29.9	9.9 [0.0, 22.3]	1.7 [0.0, 5.0]
RL, step 64, optimizer 2	25.6	17.3 [5.7, 31.8]	8.5 [2.8, 14.2]

C Reproducibility

1. `python3 scripts/m5c1_pins.py` (sources, versions, AST diff); `scripts/m5c1_verifier_audit.py`, `m5c1_roundtrip.py`, `m5c1_demonstrations.py`, `m5c1_taxonomy.py`.
2. `python3 scripts/m5c2_oracle_audit.py`; `scripts/m5c2_defect_validation.py`, `m5c2_defect_validation_extra.p`
3. `python3 scripts/m5c3_make_figures.py` regenerates all figures and tables from the stored results.
4. `python3 -m pytest tests -q` (86 tests); single cell: `python3 scripts/m5c1_probe_one.py --collection IFBench-58 --template words:vowel --probe yes`.
5. Upstream reports: `docs/upstream/` (stand-alone reproductions and `test_proposed_fixes.py`).